JavaScript Classes: Inheritance and Encapsulation

Introduction

JavaScript classes provide a structured approach to object-oriented programming. This document explores the fundamental concepts of encapsulation and inheritance through practical examples of banking account systems.

Encapsulation with Private Fields

Encapsulation is a core principle of object-oriented programming that restricts direct access to an object's internal state. In JavaScript, we use the `#` prefix to declare private fields, ensuring data integrity and controlled access.

The Account Class

Let's examine a basic implementation of an account management system:

```
class Account {
    // Private field declaration
    #bal

    constructor(initBal = 0) {
        this.#bal = initBal;
    }

    deposit(amt = 0) {
        this.#bal = this.#bal + amt;
    }

    withdraw(amt = 0) {
        this.#bal = this.#bal - amt;
    }

    get balance() {
```


Powered by SPRK Technologies

```
        return this.#bal;
    }
}
```

Key Concepts

Private Fields (`#bal`): The hash symbol designates `#bal` as a private field, making it inaccessible from outside the class. This prevents direct manipulation and enforces the use of class methods for all balance modifications.

Constructor Method: The `constructor(initBal = 0)` initializes the account with a starting balance. The default parameter `= 0` ensures that if no initial balance is provided, the account starts with zero.

Public Methods: The `deposit()` and `withdraw()` methods serve as the controlled interface for modifying the balance. They validate and process transactions while maintaining encapsulation.

Getter Method: The `get balance()` accessor provides read-only access to the private balance. Note that getters are accessed like properties, not methods.

Usage Example

```
const a = new Account(500);
console.log(a.balance);  // Access via getter
a.deposit(200);
console.log(a.balance);
```

Output:

```
500
700
```

Note:

Attempting to access `a.#bal` directly will result in a syntax error. Private fields are only accessible within the class definition.

Powered by SPRK Technologies

Inheritance and Method Overriding

Inheritance allows us to create specialized classes that extend the functionality of existing classes. The child class inherits all properties and methods from the parent while adding or modifying behavior as needed.

The SavingsAccount Class

The `SavingsAccount` extends `Account` to implement transaction limits, a common feature in savings accounts:

```
class SavingsAccount extends Account {
    #t  // Private transaction counter

    constructor(initBal) {
        super(initBal);  // Call parent constructor
        this.#t = 0;
    }

    deposit(amt = 0) {
        if (this.#t < 5) {
            super.deposit(amt);
            this.#t++;
        } else {
            console.log("Transaction limit reached");
        }
    }

    withdraw(amt = 0) {
        if (this.#t < 5) {
            super.withdraw(amt);
            this.#t++;
        } else {
            console.log("Transaction limit reached");
        }
    }
}
```


Powered by SPRK Technologies

Key Concepts

The `extends` Keyword: This establishes the inheritance relationship, making `SavingsAccount` a subclass of `Account`. The child class inherits all non-private members of the parent.

The `super` Keyword: When used in the constructor, `super(initBal)` calls the parent class constructor. This must be called before accessing `this` in the child constructor. When used in methods like `super.deposit(amt)`, it invokes the parent class implementation.

Method Overriding: The `SavingsAccount` class overrides `deposit()` and `withdraw()` to add transaction limiting logic. The overridden methods extend rather than replace the parent functionality by calling `super` methods.

Private Field Isolation: Note that `#t` is private to `SavingsAccount`, and `#bal` remains private to `Account`. Even though `SavingsAccount` inherits from `Account`, it cannot directly access `#bal`—it must use the inherited methods.

Practical Implementation

```
const s = new SavingsAccount(1000);

for (let i = 0; i < 7; i++) {
    s.deposit(500);
    console.log(s.balance);
}
```

Output:

```
1500
2000
2500
3000
3500
Transaction limit reached
```


Powered by SPRK Technologies

```
3500
Transaction limit reached
3500
```

Execution Flow Analysis

Understanding the execution sequence is crucial for working with inheritance:

1. The loop attempts seven deposit operations on the savings account.
2. For transactions 1-5, the `deposit()` method checks if `#t < 5`, calls `super.deposit()` to update the balance, and increments the transaction counter.
3. For transactions 6-7, the condition `#t < 5` evaluates to false, preventing the deposit and logging an error message.
4. The balance remains at 3500 after the transaction limit is reached.

Enhanced Error Handling

Production-ready code should implement robust error handling. Here's an improved version of the `SavingsAccount` with proper exception throwing:

```
class SavingsAccount extends Account {
    #t
    #transactionLimit = 5;

    constructor(initBal) {
        super(initBal);
        this.#t = 0;
    }

    deposit(amt = 0) {
        if (this.#t >= this.#transactionLimit) {
            throw new Error(`Transaction limit of ${this.#transactionLimit} exceeded`);
        }
        super.deposit(amt);
        this.#t++;
    }

    withdraw(amt = 0) {
        if (this.#t >= this.#transactionLimit) {
```


Powered by SPRK Technologies

```
            throw new Error(`Transaction limit of ${this.#transactionLimit} exceeded`);
        }
        super.withdraw(amt);
        this.#t++;
    }

    get transactionsRemaining() {
        return this.#transactionLimit - this.#t;
    }
}
```

Usage with Error Handling

```
const s = new SavingsAccount(1000);

try {
    for (let i = 0; i < 7; i++) {
        console.log(`Transactions remaining: ${s.transactionsRemaining}`);
        s.deposit(500);
        console.log(`Balance: ${s.balance}`);
    }
} catch (error) {
    console.error(`Error: ${error.message}`);
    console.log(`Final balance: ${s.balance}`);
}
```

Output:

```
Transactions remaining: 5
Balance: 1500
Transactions remaining: 4
Balance: 2000
Transactions remaining: 3
Balance: 2500
Transactions remaining: 2
Balance: 3000
Transactions remaining: 1
Balance: 3500
```


Powered by SPRK Technologies

```
Transactions remaining: 0
Error: Transaction limit of 5 exceeded
Final balance: 3500
```

Additional Enhancements

Input Validation

A robust implementation should validate transaction amounts:

```
class Account {
    #bal

    constructor(initBal = 0) {
        if (initBal < 0) {
            throw new Error("Initial balance cannot be negative");
        }
        this.#bal = initBal;
    }

    deposit(amt = 0) {
        if (amt <= 0) {
            throw new Error("Deposit amount must be positive");
        }
        this.#bal = this.#bal + amt;
    }

    withdraw(amt = 0) {
        if (amt <= 0) {
            throw new Error("Withdrawal amount must be positive");
        }
        if (amt > this.#bal) {
            throw new Error("Insufficient funds");
        }
        this.#bal = this.#bal - amt;
    }

    get balance() {
```


Powered by SPRK Technologies

```
        return this.#bal;
    }
}
```

Testing Validation

```
const account = new Account(1000);

try {
    account.withdraw(1500);  // Should fail
} catch (error) {
    console.error(error.message);
}

try {
    account.deposit(-100);  // Should fail
} catch (error) {
    console.error(error.message);
}
```

Output:

`Insufficient funds`

`Deposit amount must be positive`

Summary

This document covered essential object-oriented programming concepts in JavaScript:

Encapsulation: Using private fields (`#`) to protect internal state and provide controlled access through public methods and getters.

Inheritance: Creating specialized classes using the `extends` keyword to build upon existing functionality.

Method Overriding: Customizing inherited methods while leveraging parent implementations via `super`.

Constructor Chaining: Using `super()` in child constructors to properly initialize parent class state.

Error Handling: Implementing validation and throwing meaningful errors to create robust, production-ready code.

> **Note:**
>
> When designing class hierarchies, consider the principle of least privilege: expose only the minimum interface necessary for users of your class, keeping implementation details private and changeable without affecting external code.